

AgentIA — System Review, Verification and Cost

The agentic workflow, the orchestration, what is measured, and what it costs

AgentIA Engineering

2026-09-29

What this document is. A review of the system as it actually runs today: the agentic generation workflow (the LangGraph pipeline and its verification loop), the orchestration around it, the design, the verification results obtained this week, and a cost/usage report gathered from the platform's own stores — the local SQLite system of record and its MLflow mirror. It is written *after* the platform produced its first verified builds, so it separates what is measured from what is asserted. Where the two cost sources disagree, this document says so rather than picking one silently.

Contents

1	Executive summary	2
2	The agent: what it does, and how it is governed	2
2.1	The job, stage by stage	2
2.2	Autonomy with a hard stop	3
2.3	Two engines, one decision	3
2.4	Guardrails the project owns	3
2.5	The agent cannot grade its own homework	3
2.6	The prompts, the skill, and the reflector	3
3	Orchestration	4
3.1	Two entry points, one product	4
3.2	Queue, lifecycle and streaming	4
3.3	Measurement and self-improvement substrate	4
4	User interface availability	4
5	Design	5
5.1	Service inventory	5
6	Verification results	6
6.1	Coverage scaling: the reliability lever	6
6.2	The verifier is intermittently flaky	6
7	Cost and usage	7
7.1	System of record (authoritative)	7
7.2	Per-task and per-stage (real measurements)	7
7.3	MLflow mirror (and where it has drifted)	8
7.4	Why the MLflow UI can look empty	8
8	Findings and next steps	8

1 Executive summary

Four facts dominate this review.

1. **The platform now verifies its own output.** Until this week the system reported “0 of 15 sessions verified”. The cause was a one-character Docker mount defect (a private SELinux relabel concatenated onto a read-only flag, producing an invalid mount option that made every build exit before Maven ran). It is fixed, and the first verified builds are on record.
2. **The agentic workflow is real and governed.** A LangGraph state machine runs eight nodes — validator, five generation stages, sandbox, repair — with a deterministic compliance gate at every stage and a bounded repair loop. Every generation stage has two implementations, MODEL and DETERMINISTIC, dispatched once per session.
3. **Verification-guided selection is the measured reliability lever.** With a single sample the generator builds 78% of the time; sampling three candidates and keeping the first that builds raises task-level coverage to 100% at \$0.20 per reliable service.
4. **The verifier is intermittently flaky, and the cost mirror has drifted.** The same frozen workspace rebuilds 4-of-6 times under load and 8-of-8 times sequentially; the leading cause is the same mount configuration. And the MLflow mirror undercounts the system of record by $\approx$ \$1.6 (20%), so the local store — not the mirror — is the number to quote.

2 The agent: what it does, and how it is governed

At its core, AgentIA is a single autonomous worker with one job: turn a formal blueprint into a complete, compilable, test-passing Spring Boot microservice — and refuse to report success unless that is true. It is not a chatbot that suggests code; it is a governed pipeline that writes, checks, builds and verifies, and stops when it cannot. This section describes that worker in plain terms: the stages it runs, the guardrails that constrain it, and the loop that keeps it honest.

2.1 The job, stage by stage

Eight stages take a specification to a verified service. Each one has a single, reviewable responsibility, which is what makes the output auditable.

Stage	Produces	Why it matters
Validator	passes or rejects the blueprint	a bad spec is caught before a single token is spent
Scaffolder	pom.xml, configuration, entry point	the dependency allowlist and the skeleton, fixed by policy
Domain	JPA entities + immutable DTO records	the data model, in one canonical place
Service	repository, service, implementation	business logic isolated in its own layer
Controller	REST endpoints + global error handler	a uniform API surface
Test	unit tests derived from the acceptance scenarios	every declared behaviour has a test
Sandbox	compiles and runs the tests, offline	proof, not a claim
Repair	a surgical patch, then re-runs	self-correction, bounded

The order is not arbitrary: each stage builds only on artifacts the previous stages have already produced, so a decision made upstream — the package name, an entity, a contract — is never silently contradicted downstream.

2.2 Autonomy with a hard stop

The pipeline is deliberately a bounded loop, shown in Figure 2. Generation runs in one direction; then the sandbox builds. If the build fails, the repair stage reads the actual Maven error, applies a narrow patch, and builds again — up to a fixed limit (three attempts by the constitution, five in one config file, an inconsistency noted in Section 8). At the limit the session is marked **BLOCKED** and handed to a human.

That bound is the point, not a limitation: the agent may correct itself, but it may not loop indefinitely, and it may not silently hand back something it could not build. Bounded autonomy is the property a board can underwrite — the cost of a wrong answer is capped at a review, never an unbounded token burn or an unverifiable artifact.

2.3 Two engines, one decision

Every generation stage has two implementations: **DETERMINISTIC** (offline templates, zero model calls, byte-identical output) and **MODEL** (a commodity LLM, currently **deepseek-flash**, driven by a versioned instruction set). The choice is made once per session and recorded, never re-decided halfway, so a session cannot mix the two. The deterministic engine exists for cost and for audit: the same input must produce the same output when an auditor asks, and a session must be able to run with no network and no API key.

2.4 Guardrails the project owns

Before any file is written, it passes a compliance gate that merges two rule families — the constitutional test analysis and the security architecture scan. Every finding carries an attribution: **LOCAL** (this stage owns the offending file) or **ACCUMULATED** (a whole-project concern, e.g. “errors must be centralised”). A local violation rejects the candidate and spends one correction attempt; an accumulated one is recorded but never blamed on a single stage.

The key property for the board: **these are the project’s own rules, enforced deterministically — not the model’s opinion of its own output.**

2.5 The agent cannot grade its own homework

The decisive design choice is that the acceptance test is *not* written by the model. After every stage, the platform injects its own persistence contract test — a Spring context boot with `ddl-auto=validate` against the generated `schema.sql` — then compiles and runs everything inside a sealed Docker container with no network and a read-only dependency cache. The model never sees this test, so it cannot write to it.

This matters because the model’s own unit tests mock the persistence layer out: a generated service whose schema contradicts its entities would pass every model-written test and fail the platform’s. That class of defect has been caught here, and it is exactly the gap this design closes. Test counts are read from the build’s own output, never fabricated, and a build that reports success without running any test is recorded as not verified.

2.6 The prompts, the skill, and the reflector

Three pieces of authored text drive the model, and they are worth naming because they are the platform’s real, reviewable “program”:

- **The five stage prompts** (`resources/instructions/*.md`). One per stage, each in four fixed sections — *Technology contract*, *Rules*, *Output contract*, *Prohibitions*. They are injected verbatim ahead of every model request and versioned content-addressed, so an artifact records exactly which instruction text produced it.
- **The skill** (`resources/skills/layer_architecture.md`). A single document encoding the layering discipline: one class per layer; depend only on the layer immediately below; never import a repository from a controller. The skill slot currently points at nothing — two further skill files are empty and the active pointer is absent — so the skill is defined but not yet wired in.
- **The reflector prompt** (`scripts/skillopt/prompts/analyst_error.md`, adapted from Skillopt, Appendix C.2.1). This is the text that would turn recorded diagnostics into skill edits: it consumes the rule histogram, per-stage attribution, the build exit code, the terminal status and the verification-fallback flag — the diagnostic channel, not a binary verdict.

3 Orchestration

3.1 Two entry points, one product

The system exposes two ways to start a session, and this review confirms both now reach the model and both verify the workspace:

- **Graph path** — `POST /sessions` enqueues a session whose worker runs the compiled graph end to end, credentials threaded through the `X-LLM-API-Key` header.
- **Sequential path** — `POST /sessions/quick-start` with `autoRun`, and `/orchestrator/pipeline/run`, drive `pipeline_runner`, which runs the same generation stages, the same hermetic verification, a static security audit, and DevOps/export phases.

The sequential path previously generated code and *never compiled it*; it now runs the same verification seam as the graph, so the two paths cannot drift.

3.2 Queue, lifecycle and streaming

A FIFO queue (`queue_service.py`) caps concurrency at two simultaneous sandbox executions. The lifecycle (`lifecycle_service.py`) walks `REQUIREMENTS` → `ARCHITECTURE` → `DATA_MODEL` → `CODE_TESTS` → `SECURITY_AUDIT` → `DEVOPS` → `EXPORT` → `COMPLETED`, with in-flight pause, resume and cancel and live Server-Sent-Events streaming to the monitor screen.

3.3 Measurement and self-improvement substrate

Two further layers sit alongside generation. The diagnostic instrument (`conformance_diagnostics.py`) is a deterministic, rule-attributed scorer that persists one durable record per session and produces a corpus report. The skill-optimisation loop (features 014–015) is *built* — collect, reflect, apply, a strict-improvement gate, per-skill contribution by leave-one-out, and eviction with an evidence floor — but it is deliberately not run, for the reasons stated in Section 6.

4 User interface availability

The product surface is a React/Vite single-page application (the earlier Streamlit app is retained as *frontend-legacy*). It is organised as a ten-tab workspace with Spanish labels, tabulated below; a lifecycle stepper mirrors the backend phase machine, and architecture and entity-relationship diagrams render through a Mermaid viewer.

Tab	What it exposes
-----	-----------------

0. Resumen	executive dashboard; 1-click quick start; Auto-Pilot vs step-by-step mode
1. Requisitos	BDD user stories (Given/When/Then) and spec.md export
2. Arquitectura	4-layer architecture, component catalogue, REST contracts (Mermaid)
3. Modelos & SQL	JPA entities, DTO records, schema.sql, ER diagram
4. Blueprints	drag-and-drop spec.md / JSON blueprint ingestion and validation
5. Monitor Live	live logs over Server-Sent Events, phase progress, pause/resume/cancel
6. Código & Fix	code explorer, tests, repair diffs, manual intervention for blocked sessions
7. Calidad SAST	static secrets/CVE/architecture audit, quality gate, 1-click remediation
8. DevOps & Demo	Dockerfile, compose, CI/CD, k8s manifests, local deploy, smoke test, REST console
9. Entrega Git	ZIP download and atomic Git publish

Beyond the tabs: an LLM settings drawer (provider: Gemini, Groq, OpenAI, DeepSeek or offline mock; API key; model; connection verification), authentication on the login screen, a dark/light theme toggle, and a session list with status. The UI mirrors the backend's honest three-state verification (verified / verified with findings / not evaluable), following a frontend audit that removed several places where the interface reported outcomes that had not occurred (a fabricated smoke-test “UP” verdict, a REST console that invented 200/201/204 responses, and a CRUD form that invented a created record).

5 Design

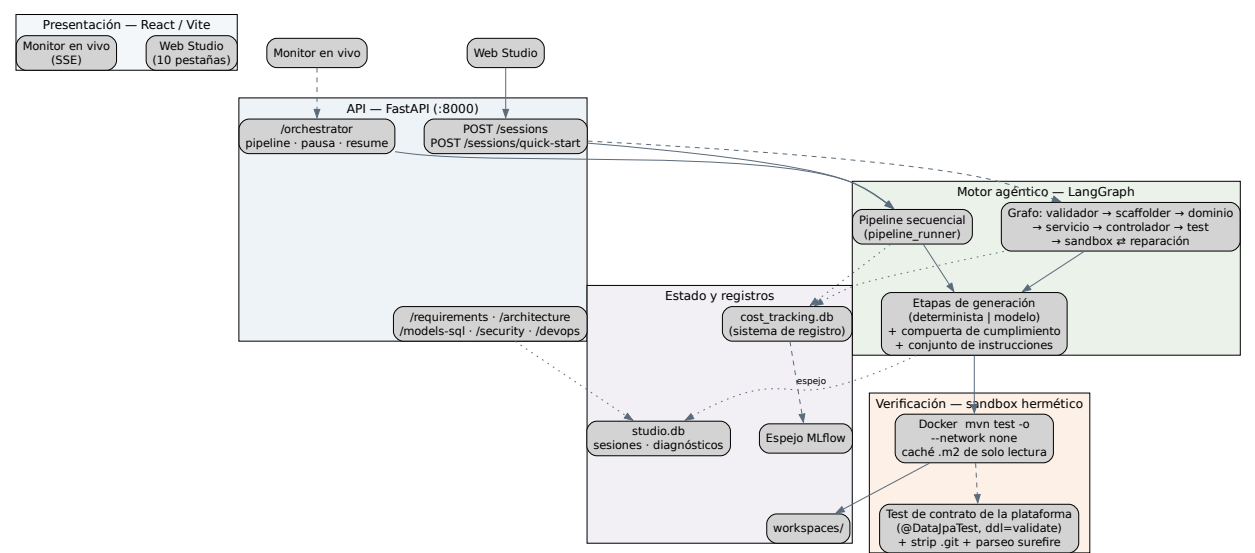


Figure 1: System architecture: the React studio over the FastAPI API, the LangGraph agentic engine, the hermetic verification sandbox, and the state/cost stores.

5.1 Service inventory

Service	Role
requirements_service	BDD story decomposition and spec synthesis

architecture_service	4-layer architecture and component catalogue
model_sql_service	JPA entities, DTOs, schema/data SQL, ER diagram
test_analysis_service	static constitutional-rule analyser (family A)
security_service	SAST, secrets, architecture conformance (family B)
conformance_diagnostics	deterministic rule-attributed conformance scorer
corpus_report	batch/corpus measurement report
workspace_verification	strip VCS, inject contract test, run the build
verification_selection	best-of- k : generate, verify, stop at first pass
platform_verification	platform-authored persistence contract test
skill_contribution	leave-one-out per-skill contribution
llm_factory	provider detection and model-client construction
structured_output	schema-constrained model output
injection_guard	prompt-injection defence for the reflector
queue_service /	concurrency and phase orchestration
lifecycle_service	
docker_service / devops /	container deploy, CI, ZIP, publish
export / git	
cost recording + mlflow sink	per-call and per-session cost, mirror

6 Verification results

6.1 Coverage scaling: the reliability lever

Verification-guided selection — generate k candidates, verify each, keep the first that passes — is measured here, not borrowed from another task class. The control arm of this week’s experiment is 6 tasks, each resampled 3–4 times with the same instructions and model; the coverage formula is $\text{coverage}(k) = 1 - (1 - p)^k$ (*Large Language Monkeys*, [arXiv:2407.21787](#)).

Metric	Value
pass@1 (single sample)	78.3%
coverage $k=1$	83.3%
coverage $k=2$	83.3%
coverage $k=3$	100.0%
coverage $k=4$	100.0%

At \$0.065 per generated service, three candidates cost **\$0.20 per reliable service**. Caveats, stated not smoothed: $n=6$ tasks, one model, one provider; this is a demonstration of the lever, not a benchmark, and the direction (coverage is monotone in k) is robust while the 100% is “6 of 6 tasks”.

6.2 The verifier is intermittently flaky

The same frozen workspace, rebuilt under load earlier, passed 4 of 6 times; rebuilt sequentially, two workspaces passed 8 of 8 each. The failure, when it occurs, is a surefire `PluginContainerException` on a `surefire-shared-utils` foreign import — *after* every test has passed.

The flake is therefore not a per-workspace code defect. It is intermittent and load-correlated, and the leading hypothesis is the mount configuration: `DOCKER_MOUNT_SUFFIX=:Z` is a *private* SELinux relabel, correct for a per-session workspace but wrong for a *shared read-only Maven cache* mounted by many concurrent containers. Two containers relabelling one shared source with `:Z` is a known cause of exactly this class of intermittent class-realm failure. The proposed fix is to apply `:Z` to the workspace only and leave the cache unrelabelled (or `:z`), confirmed by a repeatability gate.

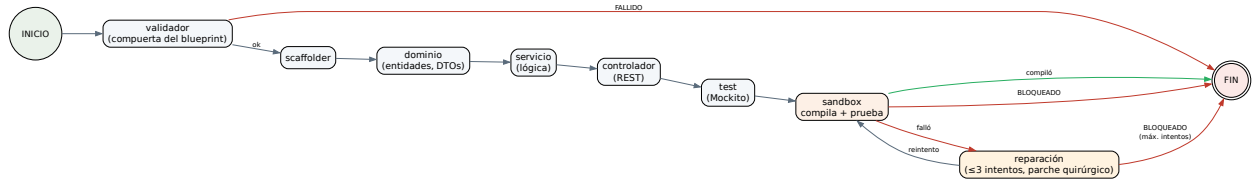


Figure 2: The generation state machine: a linear spine with one bounded repair-sandbox cycle.

Why this gates everything downstream. A verifier that flips $\approx 33\%$ of runs cannot serve as an admission oracle: coverage would over-promise, an admission gate would promote noise, and Ratchet’s theorem ([arXiv:2605.22148](https://arxiv.org/abs/2605.22148)) says a judge this noisy retires nothing at any sample size. Making the oracle deterministic is therefore the prerequisite for every further measurement — including the skill-optimisation loop, which stays closed behind it.

7 Cost and usage

Costs are recorded per call and per session in the local SQLite store (`backend/cost_tracking.db`), which is the **system of record**, and mirrored into MLflow (`mlflow.db`). The two are compared here, not merged.

7.1 System of record (authoritative)

Scope	Calls	Cost (USD)	Output tokens
Polarity experiment (p4)	750	6.58	—
Frozen-five baseline (corpus)	120	0.63	—
Hard ten	50	0.64	—
Probes / single sessions	≈ 250	≈ 0.16	—
Total	1172	8.01	12.28M

All 1172 calls were `deepseek-flash`. Total input tokens 6.45M, output tokens 12.28M. The single dominant cost is the instruction-polarity experiment, which is a finished measurement (a null), not an ongoing drain.

7.2 Per-task and per-stage (real measurements)

Across the ten-service “hard” corpus, a generated service costs **\$0.052–\$0.077**, for a corpus total of **\$0.6357** over 50 calls — roughly **\$0.005 per model call**, and cost scales with blueprint complexity (payload and output tokens), not with retries.

Stage (one service)	In tok	Out tok	Cost
Scaffolder	2,200	6,620	\$0.0040
Domain	2,669	1,759	\$0.0012
Service	3,250	5,166	\$0.0034
Controller	3,925	4,210	\$0.0029
Test	5,711	9,108	\$0.0061
Total	17,755	26,863	\$0.0176

The test stage is the most expensive because its output is the largest; the scaffold and domain stages are the cheapest. No single stage dominates, so cost is predictable and linear in service size.

7.3 MLflow mirror (and where it has drifted)

Experiment	Runs	Cost (USD)	In tok	Out tok
Default (legacy)	877	5.41	4.63M	8.20M
agentia (current)	114	0.98	0.78M	1.52M
Mirror total	991	6.39	5.40M	9.72M

The mirror has drifted. MLflow totals **\$6.39** against the system of record’s **\$8.01** — an undercount of $\approx \$1.62$ (20%), with ≈ 181 fewer calls and token totals that also disagree. The mirror is split across a legacy and a current experiment, and only per-call metrics are being written (session aggregates are absent). The earlier “the mirror works and it agrees” claim is no longer true for the current data. Until the sink is repaired, the local store is the only number to quote, and a cost report generated from MLflow alone would understate spend.

7.4 Why the MLflow UI can look empty

Two independent causes. First, the data lives in `mlflow.db` (SQLite); the filesystem store `mlruns/` — which `mlflow ui` opens *by default* — is empty, so a bare `mlflow ui` shows nothing. The server must be started with `-backend-store-uri sqlite:///mlflow.db`. Second, even in the right store the records are in the wrong experiment: all 168 session aggregates and 623 of the per-call runs are in the legacy `Default` experiment, while the `agentia` experiment (which the configuration names) holds 0 session records and 119 per-call runs. The sink’s experiment targeting does not reach session records.

8 Findings and next steps

1. **Make the verifier deterministic before spending anything more.** Split the mount suffix (workspace `:Z`, shared cache unrelabelled) and add a repeatability gate: one frozen workspace must rebuild N/N times.
2. **Make the measure able to move.** The conformance instrument reads 0 on failing builds; surface the repair parser’s failure classes as accumulated findings so a build failure is attributable.
3. **Use verification-guided selection as the reliability mechanism now.** It is measured ($\$0.20/\text{service}$ at $k=3$), it needs no training, and it is the lever the literature ranks first.
4. **Repair the cost mirror.** Session-level mirroring is not populating; restore it or the MLflow UI understates spend.
5. **Resolve the repair-cap inconsistency** (constitution 3 vs `config.py` 5).
6. **Leave the skill-optimisation loop closed** until items 1 and 2 pass their gates; the instruction-polarity experiment that was its most promising input is a null (prohibition vs control: 1 win, 3 losses, 2 ties).